

Jour 2 : Les data pour le Machine Learning, collecte et preprocessing

Bonjour bonjour ! Hier, on a monté un pipeline qui apprend et qui prédit, et on a même ouvert l'Arène. Ça tournait nickel... parce que vous aviez des données déjà propres, livrées clé en main par scikit-learn.

Mais... dans la vraie vie, ça n'existe pas. La donnée arrive sale ! Des trous, des valeurs aberrantes, des colonnes en texte que l'algo ne sait pas lire, des variables qui disent dix fois la même chose. Et voilà le secret le moins glamour du métier : un data scientist passe environ **80% de son temps** non pas sur l'algo, mais à préparer la donnée.



(oh non encore un gif)

Aujourd'hui c'est ménage 😊 ...

Le battle cry du jour : **sans data propre, le meilleur algo du monde donne de la bouillie**. Aujourd'hui, on apprend à inspecter une donnée, à la comprendre, et à la nettoyer de bout en bout. À la fin de l'après-midi, vous aurez transformé un vrai jeu de données crasseux en un jeu propre, prêt à modéliser. Let's go.

1 - Révisions rapides

Trente secondes pour rebrancher les neurones d'hier, on va construire une couche par dessus.

- **Les familles d'apprentissage** : supervisé (on a les réponses, les "labels") contre non-supervisé (on cherche une structure cachée). La PCA de cet après-midi est du non-supervisé.
- **Le pipeline scikit-learn** : charger, séparer (train/test), entraîner (`.fit`), prédire (`.predict`), mesurer. Le même squelette pour tous les algos.

- **CRISP-DM** : les 6 phases d'un projet data. Hier on a posé "Compréhension métier" et "Compréhension des données". Aujourd'hui, on plonge dans la phase 3, la **préparation des données**, celle qui mange 80% du temps.
- **Le piège entrevu hier** : la régression logistique qui râlait sans mise à l'échelle, et la fuite de données quand on ajuste un scaler sur tout le jeu. On formalise tout ça aujourd'hui.

Petit réveil collectif : qui se souvient de la différence entre un hyperparamètre et un paramètre ?

(Cette fois ci je mets la réponse plus bas, pour pas qu'on se spoil)

Allez on enchaîne

Planning de la Journée

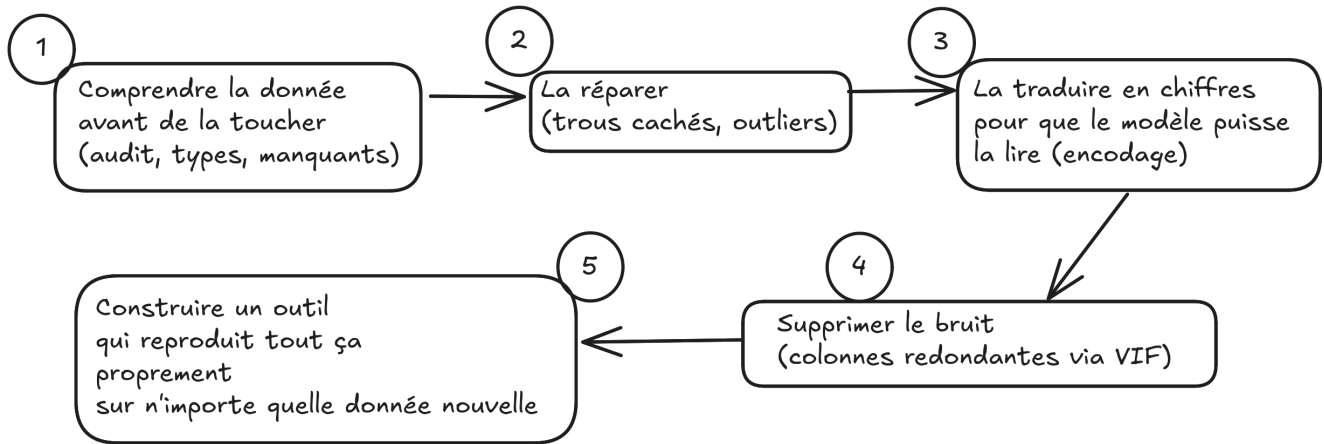
1. **Les trois familles de données** : structurées, semi-structurées, non structurées, et le monde de l'IoT et du streaming
2. **La nature statistique des données** : qualitatif contre quantitatif, et comment transformer du texte en chiffres (l'encodage)
3. **Les valeurs manquantes** : les détecter, et choisir comment les corriger sans mentir à son modèle
4. **Les valeurs aberrantes** : repérer les outliers (boxplot, IQR, z-score) et décider quoi en faire
5. **Corrélations et multicolinéarité** : voir les variables qui bougent ensemble, et pourquoi deux jumelles déstabilisent un modèle
6. **Les variables les plus discriminantes** : trier les features par pouvoir prédictif
7. **La réduction de dimensions** : la PCA, comment compresser 30 colonnes en 2 sans tout perdre
8. **L'après-midi (À votre tour)** : nettoyer un vrai dataset client sale de bout en bout, en faire une boîte à outils réutilisable, puis la crash-tester sur la data de votre choix

Le matin suit toujours le même rythme : je pose le concept, je montre, vous changez un paramètre et vous observez, on confronte avec le voisin (quand possible). Et coupez-moi la parole ;)

Réponse à la question au-dessus du planning : le **paramètre**, c'est ce que le modèle apprend tout seul pendant l'entraînement (les coefficients d'une régression, les seuils de coupure d'un arbre). Vous n'y touchez pas, c'est `.fit()` qui les fixe. L'**hyperparamètre**, lui, c'est ce que VOUS réglez AVANT d'entraîner (le `n_neighbors=3` du KNN d'hier, le nombre d'arbres d'une forêt). Moyen mnémo : l'hyper est au-dessus, vous le décidez ; le paramètre est dedans, le modèle le trouve.

En résumé, on va apprendre à faire le vrai job de nettoyage d'un data scientist :

Le Vrai Job d'un Data Scientist



2 - Les trois familles de données

Avant de nettoyer quoi que ce soit, il faut savoir à quoi on a affaire. Toutes les données ne se ressemblent pas, et la première question d'un projet, c'est : *"c'est quoi, cette donnée, et est-ce que je sais la mettre dans un tableau ?"*.

L'essentiel

On classe la donnée du monde en trois grandes familles, selon à quel point elle est déjà rangée.

L'échelle, en résumé : Données structurées > Semi-structurées > Non-structurées

Données structurées : tout ce qui rentre dans un tableau bien carré, lignes et colonnes, chaque colonne avec un type fixe. Une base de données clients, un export Excel de ventes, un CSV de transactions. C'est le format roi du ML classique, et c'est presque exclusivement ce qu'on manipule cette semaine.

Données semi-structurées : ça a une structure, mais souple, pas un tableau rigide. Un fichier [JSON](#) renvoyé par une API, du XML, un document dans une base [NoSQL](#) (ici donc des bases qui ne stockent PAS la donnée en tableaux SQL bien carrés, justement pensées pour de la donnée souple à structure variable, comme MongoDB). Les champs existent, mais ils peuvent varier d'un enregistrement à l'autre (un client a un deuxième numéro de téléphone, un autre ne l'a pas).

C'est quoi JSON ? "JavaScript Object Notation", un format texte pour échanger des données structurées en paires clé-valeur, lisible par un humain comme par une machine. C'est le format de réponse standard de la quasi-totalité des API web.

Vous connaissez XML ?

Données non structurées : tout le reste, le gros morceau. Du texte libre (un avis client, un mail), des images, du son, de la vidéo. Aucune ligne, aucune colonne. C'est là que vit le Deep Learning, parce qu'il faut des techniques lourdes pour en extraire du sens.

Un chiffre qui circule beaucoup dans l'industrie : environ **80 à 90% de la donnée d'une entreprise est non structurée**. Mais paradoxe utile, l'immense majorité des problèmes business résolus au quotidien (churn, c'est-à-dire le départ ou la résiliation d'un client ; scoring ; prévision de ventes) tournent sur de la donnée structurée. D'où notre focus tabulaire.

Famille	Exemple	Format typique	Qui s'en occupe
Structurée	Table clients, ventes	CSV, SQL, Excel	ML classique (nous)
Semi-structurée	Réponse d'API, logs	JSON, XML, NoSQL	Data engineering
Non structurée	Texte, image, son	.txt, .jpg, .wav	Deep Learning

Triez les sources : pour chacune, structurée, semi-structurée ou non structurée ?

- un fichier CSV des notes d'élèves
- le flux de tweets en temps réel sur un hashtag
- la réponse JSON de l'API météo
- une photo de votre chat
- une table SQL de commandes

Comparons les réponses dans la salle

IoT et streaming : la donnée qui ne s'arrête jamais

Jusqu'ici on a imaginé un fichier qu'on charge d'un coup. Mais une part énorme de la donnée moderne arrive **en flux continu**, et ça change la donne.

L'IoT (Internet of Things, l'Internet des objets) : des milliards de capteurs connectés qui crachent de la donnée en permanence. Un thermostat connecté, un bracelet Fitbit, un capteur de vibration sur une machine d'usine, une voiture qui remonte sa position. Chacun envoie des petits paquets de mesures, sans arrêt.

Et le "streaming" de données ? Alors là on parle pas de Netflix : ici c'est de la donnée qui arrive en continu, événement après événement, qu'on doit traiter au fil de l'eau plutôt qu'en un seul gros fichier. L'outil emblématique du domaine est [Apache Kafka](#), une sorte de tapis roulant géant pour faire transiter des millions d'événements par seconde.

Aussi [RabbitMQ](#) très utilisé

Pourquoi ça nous concerne, nous qui faisons du ML sur des tableaux ? Parce que ça relie deux mondes croisés hier :

- En **batch** (par lots), on accumule la donnée du flux, on la fige dans une table, et on entraîne dessus tranquillement. C'est notre semaine.
- En **online** (en ligne), le modèle se met à jour au fil du flux. Indispensable pour la détection de fraude en temps réel ou la pub.

La maintenance prédictive vue hier, c'est exactement ça : un capteur IoT mesure la vibration d'un moteur en continu, le flux est stocké, et un modèle apprend à prédire la panne avant qu'elle arrive. La donnée naît en streaming, on la range en tabulaire, et là on retombe sur tout ce qu'on sait faire.

Approfondissons

Une notion de fond qui sépare les amateurs des pros : la **"tidy data"** (donnée bien rangée), formalisée par [Hadley Wickham](#), une figure du monde de la data. La règle est bête mais violée partout : **une ligne = une observation, une colonne = une variable, une cellule = une valeur**.

Ça paraît évident, et pourtant la donnée réelle viole ça constamment : deux infos collées dans une même cellule ("Paris, 75001"), une variable étalée sur plusieurs colonnes (une colonne par mois au lieu d'une colonne "mois"), des en-têtes qui sont en fait des valeurs. Une grosse partie du "nettoyage" consiste juste à remettre la donnée au format tidy avant de pouvoir faire quoi que ce soit. Gardez ce réflexe : avant de modéliser, demandez-vous toujours "ma donnée est-elle tidy ?".

💡 **Bonus perso** : ouvrez un export de votre vie numérique (relevé bancaire en CSV, export Strava, historique Spotify). Regardez : est-ce structuré ? Tidy ? Combien de colonnes mélangent deux infos ? Vous venez de faire le premier geste du métier sur vos propres données.

Vous avez d'autres idées de choses perso (mais pas trop) qu'on pourrait exporter ?

Pour les exports de playlists Spotify en CSV (liens à retester / revérifier) : [exportify.net](#) , [chosic.com](#)

On sait ranger la donnée par familles. Maintenant, zoomons dans un tableau structuré : toutes les colonnes ne se valent pas, et leur nature statistique décide de tout ce qu'on a le droit de leur faire.

3 - La nature statistique des données, et l'encodage

Voici un tableau de clients.

Une colonne "âge", une colonne "ville", une colonne "satisfaction (de 1 à 5)", une colonne "a résilié (oui/non)".

Question "piège" : peut-on calculer la moyenne de la colonne "ville" ? Eh bien évidemment non. Mais pourquoi ? Parce que ces colonnes n'ont pas la même **nature statistique**, et c'est ça qui dicte le traitement.

Le coeur du sujet

On distingue deux grandes natures, chacune coupée en deux.

Données qualitatives (catégorielles) : des catégories, pas des quantités. On ne fait pas de maths dessus.

- **Nominales** : des étiquettes sans ordre. La ville, la couleur, le pays. "Paris" n'est ni plus grand ni plus petit que "Lyon".
- **Ordinales** : des catégories avec un ordre naturel, mais sans distance mesurable. "Petit < Moyen < Grand", ou une note de satisfaction "Mécontent < Neutre < Satisfait". On sait que c'est ordonné, mais l'écart entre "Petit" et "Moyen" n'est pas chiffrable.

Données quantitatives (numériques) : des nombres, et là les maths ont du sens.

- **Discrètes** : des valeurs qu'on compte, souvent entières. Le nombre d'enfants, le nombre de commandes. Pas de "2,5 enfants".
- **Continues** : des valeurs qu'on mesure, sur une échelle infinie. La taille, le prix, la température. Entre 1,70 m et 1,71 m, il y a une infinité de valeurs.

Pourquoi ça compte autant ? Parce que le traitement dépend de la nature :

- Une **moyenne** n'a de sens que sur du quantitatif. Sur du nominal, on regarde le **mode** (la catégorie la plus fréquente).
- Une valeur manquante quantitative se remplace souvent par la médiane ; une valeur manquante nominale, par le mode.
- Et surtout, un algo de ML ne sait manger que des **nombres**. Les colonnes qualitatives doivent donc être transformées. C'est l'encodage, juste en dessous.

Typez les colonnes : chargeons un vrai dataset, le naufrage du Titanic, livré avec [seaborn](#) (la bibliothèque de visualisation statistique bâtie sur [matplotlib](#), la lib de graphiques de référence en Python). On le manipule avec [pandas](#), la boîte à outils des tableaux de données en Python (ses fameux "DataFrames", l'équivalent d'un Excel piloté en code). On garde ce dataset toute la matinée.

```
import seaborn as sns
import pandas as pd

# Le dataset Titanic : un passager par ligne, a-t-il survécu ou non
df = sns.load_dataset("titanic")

# .dtypes donne le type technique de chaque colonne (int, float, object...)
print(df.dtypes)
print("\nForme (lignes, colonnes) :", df.shape)
print("\nAperçu :")
print(df[["survived", "sex", "age", "pclass", "fare", "embarked"]].head())
```

Devrait afficher quelque chose comme :

```
survived      int64
pclass        int64
sex           object
age           float64
...
Forme (lignes, colonnes) : (891, 15)
```

	survived	sex	age	pclass	fare	embarked
0	0	male	22.0	3	7.2500	S
1	1	female	38.0	1	71.2833	C
2	1	female	26.0	3	7.9250	S

Attention à ne pas confondre le **type technique** (ce que voit pandas : `int64`, `object`) et la **nature statistique** (ce que la variable veut dire). `pclass` est stockée en `int64`, mais c'est une variable **ordinaire** (1re, 2e, 3e classe), pas une vraie quantité : la classe 3 n'est pas "trois fois" la classe 1. À vous, l'humain, de trancher la nature ; pandas ne devine pas.

Pour chacune, dites la nature : `sex`, `age`, `pclass`, `fare`, `survived`.

`pclass` et `survived`, sont les deux pièges.

L'encodage : transformer du texte en chiffres

Un modèle ne lit pas "male" ou "female", il lit des nombres. Il faut donc **encoder** les colonnes qualitatives. Et le choix de la méthode dépend, justement, de la nature nominale ou ordinale. C'est là que la distinction du dessus devient concrète.

Pour du nominal : le [One-Hot Encoding](#).

Avant de creuser ce concept, comprenons ce que veut dire **One-hot**. Ça vient de l'électronique numérique.

Dans un registre de bits,

=> le "hot" = actif/allumé (1),

=> le "cold" = inactif (0).

"One-hot" c'est donc exactement **un** bit est "chaud" (1), tous les autres sont "froids" (0). C'est le pattern utilisé pour encoder des états mutuellement exclusifs dans les circuits logiques, bien avant le ML.

One Hot Encoding			
color	color_red	color_blue	color_green
red	1	0	0
green	0	0	1
blue	0	1	0
red	1	0	0

Du coup vous l'aurez compris, ce qu'on fait c'est qu'**on crée une colonne binaire (0/1) par catégorie**. C'est la méthode par défaut, et la seule correcte pour du nominal (pour des étiquettes sans ordre, du coup. Du nominal.).

```
# get_dummies : la façon pandas de faire du one-hot, parfaite pour explorer
encoded = pd.get_dummies(df["embarked"], prefix="port")
print(encoded.head())
```

À savoir, dans le dataset, `df["embarked"]` contient des valeurs texte : "S", "C", ou "Q" (qui sont les ports d'embarquement du Titanic).

Devrait afficher :

```
port_C  port_Q  port_S
0   False   False   True  # ← passager 0 a embarqué à S
1    True   False  False  # ← passager 1 a embarqué à C
2   False   False   True  # ← passager 2 a embarqué à S
```

Pour chaque passager : une seule colonne est True, les deux autres sont False. C'est ça le one-hot : une seule "lampe allumée" à la fois.

Le port "S" devient `port_S=1` et les autres à 0.

Mais attends, pourquoi pas juste S=0, C=1, Q=2 ?

Un modèle ML voit des chiffres, pas du texte ! Si on code S=0, C=1, Q=2, le modèle va faire des maths avec ça : il va croire que C est "à mi-chemin" entre S et Q, ou que Q est "deux fois plus grand" que C.

Sauf que ces ports n'ont aucune relation de grandeur ou de distance entre eux. Ce sont des catégories indépendantes.

Du coup attention à ne pas inventer un ordre et des distances qui n'existe pas sans faire exprès ! **C'est l'erreur de débutant numéro un sur les catégorielles.**

Là c'est c'est comme coder Rouge=1, Bleu=2, Vert=3 puis après, demander à quelqu'un de calculer la moyenne des couleurs. Ça n'a aucun sens, mais les chiffres permettent de le faire quand même et c'est clairement ça le problème.

Bref, le one-hot évite ça du coup : chaque port a sa propre colonne, le modèle ne peut pas inventer de relation entre eux.

Pour du ordinal : l'Ordinal Encoding.

Ok dans Ordinal, on a "ordre". Là, l'ordre existe vraiment, donc on a le droit de le coder par des entiers croissants.

Ci-dessous, prenons l'exemple la classe de billet que chaque passager a choisi (un peu comme en avion, 1re classe, 2è classe, 3è classe)

En prenant un array de passagers,

- On va dire que `ordre = ["First", "Second", "Third"]`, autrement dit First < Second < Third.
- Puis transformer chaque valeur en son indice dans notre liste `fit_transform(classe)` : First → 0, Second → 1, Third → 2.
- Et enfin, utiliser `ravel()`. La fonction `fit_transform` retourne un tableau 2D (une colonne), et `ravel()` va l'aplatir en 1D pour l'affichage.

Avec [scikit-learn](#), la version propre et réutilisable :

```
from sklearn.preprocessing import OrdinalEncoder

# On impose l'ordre nous-mêmes : c'est tout l'intérêt
ordre = ["First", "Second", "Third"]
encoder = OrdinalEncoder(categories=ordre)

classe = df[["class"]].dropna()
classe_encodee = encoder.fit_transform(classe)
print(classe_encodee[:5].ravel())
```

Devrait afficher :


```
[2. 0. 2. 0. 2.]
```

passager numéro 0 → 2. → Third (3è classe)

passager numéro 1 → 0. → First (1re classe)

passager numéro 2 → 2. → Third (3è classe)

...

First devient 0, Second 1, Third 2 : l'ordre est respecté, et c'est légitime ici parce que la classe EST ordonnée.

Bref !

Le réflexe à graver :

- nominal => One-Hot Encoding
- ordinal => Ordinal Encoding

Si vous codez du nominal avec des entiers, vous mentez à votre modèle.

Un piège pratique du One-Hot :

Si une colonne a 5000 catégories différentes (par exemple un code postal ça a énormément de catégories, idem pour un identifiant produit), là vous créez 5000 colonnes. C'est l'**explosion de dimensions**, et ça rejoint directement le problème qu'on attaque en fin de matinée avec la PCA. Pour ces cas, il existe d'autres encodages (target encoding, par exemple), mais retenez surtout le signal d'alerte.

Le piège de la colinéarité

Une subtilité qui revient (par exemple en entretien) : le **piège de la colinéarité du One-Hot** (la "[dummy variable trap](#)").

Rappel : **La colinéarité c'est quoi ?** deux colonnes qui portent la même information. Ex : si tu connais `port_C` et `port_Q`, tu sais automatiquement `port_S` (c'est forcément 1 si les deux autres sont 0). Le modèle a une colonne inutile (car on la devine / déduit), voire ça crée des instabilités mathématiques.

Le principe, c'est que si on crée une colonne par catégorie, la dernière sera toujours déductible des autres. Dans notre exemple à trois colonnes, avec C, Q, S : si ce n'est ni C ni Q, c'est forcément S.

Ces colonnes sont donc parfaitement redondantes, et pour certains modèles linéaires *ça pose exactement le problème de multicolinéarité* qu'on verra en section 6.

C'est pour ça que `get_dummies` a un paramètre `drop_first=True` : celui-ci va supprimer la première colonne, car les deux restantes suffisent à tout reconstruire.

Pour un arbre ou une forêt, ça n'a aucune importance ; pour une régression, oui.

=> L'arbre / forêt ne calcule pas de coefficients et pose juste des questions binaires ("port_S = 1 ?"). Ici la colonne redondante ne perturbe rien et est ignorée naturellement

=> Par contre la régression calcule des coefficients pour chaque colonne. **Elle cherche UN jeu de coefficients optimal.** Si deux colonnes sont redondantes alors les coefficients deviennent instables : plusieurs combinaisons donnent le même résultat, le modèle sait pas trancher, il en existe une infinité qui donnent le même résultat. Le modèle ne peut pas choisir, donc les coefficients deviennent arbitraires et ininterprétables. Encore une preuve que le bon preprocessing dépend de l'algo qui suit.

Pour creuser : la doc officielle [scikit-learn sur le preprocessing](#) couvre tous les encodeurs et scalers avec des exemples. À garder en favori, c'est la référence que vous rouvrirez toute votre carrière.

Nos colonnes sont typées et on sait les rendre lisibles par un modèle. Sauf qu'on a triché : on a fait `.dropna()` pour éviter les trous. Justement, parlons-en, de ces trous.

4 - Les valeurs manquantes

Vous avez peut-être remarqué le `.dropna()` glissé dans le code d'avant. Je vous ai caché un truc : la colonne `age` du Titanic est pleine de trous, et la colonne `deck` est quasiment vide. C'est la réalité de TOUTE donnée réelle. Un capteur qui tombe en panne, un client qui ne remplit pas un champ, une fusion de fichiers ratée : les trous sont partout. Et un algo qui rencontre un trou, en général, plante.

Ce qu'il faut tenir

Étape 1 : détecter. On commence toujours par mesurer l'ampleur des dégâts.

```
import seaborn as sns

df = sns.load_dataset("titanic")

# .isna() repère les trous, .sum() les compte par colonne
manquants = df.isna().sum()
# On affiche aussi le pourcentage, plus parlant que le brut
pourcentage = (df.isna().mean() * 100).round(1)

resume = pd.DataFrame({"manquants": manquants, "pourcent": pourcentage})
print(resume[resume["manquants"] > 0].sort_values("pourcent", ascending=False))
```

Devrait afficher quelque chose comme :

	manquants	pourcent
deck	688	77.2
age	177	19.9
embarked	2	0.2
embark_town	2	0.2

Le pourcentage change tout :

- `deck` à 77% de trous : on jette probablement la colonne, elle est inexploitable.
- `age` à 20% : précieux, on va vouloir la sauver.
- `embarked` à 0,2% : deux lignes, on peut presque les ignorer.

La décision dépend du taux, pas juste du nombre.

Étape 2 : corriger. Trois grandes stratégies, de la plus brutale à la plus fine.

La suppression. On vire les lignes (ou les colonnes) avec des trous.

```
df_sans_lignes = df.dropna(subset=["age"])    # vire les lignes sans âge
df_sans_colonne = df.drop(columns=["deck"])  # vire la colonne deck entière
```

Simple, mais dangereux : supprimer 20% des lignes parce qu'il manque l'âge, c'est jeter 20% de votre information. Et si les trous ne sont pas aléatoires (voir plus bas), vous introduisez un biais.

L'imputation simple. On remplit les trous avec une valeur calculée : la moyenne, la médiane (pour du quantitatif), ou le mode (pour du qualitatif). L'outil propre est le [SimpleImputer](#) de scikit-learn :

```
from sklearn.impute import SimpleImputer
import numpy as np

age = df[["age"]]

# La médiane, plus robuste que la moyenne face aux valeurs extrêmes
imputer = SimpleImputer(strategy="median")
age_rempli = imputer.fit_transform(age)

print("Trous avant :", int(age.isna().sum().iloc[0]))
print("Trous après :", int(np.isnan(age_rempli).sum()))
print("Valeur utilisée (médiane) :", round(imputer.statistics_[0], 1))
```

Devrait afficher :

```
Trous avant : 177
Trous après : 0
Valeur utilisée (médiane) : 28.0
```

NumPy, ce `np` ? [NumPy](#) est la brique de calcul numérique de tout l'écosystème Python : des tableaux de nombres ultra-rapides sur lesquels pandas et scikit-learn sont eux-mêmes bâtis. Ici, `np.isnan` sert juste à repérer les cases vides.

Pourquoi la médiane plutôt que la moyenne ? Parce que la médiane résiste aux valeurs extrêmes (souvenez-vous, hier, le salaire du PDG qui fait exploser la moyenne mais pas la médiane). Sur de l'âge, ça change peu ; sur des revenus, énormément.

L'imputation par KNN. Plus maligne : pour remplir le trou d'une ligne, on regarde les lignes qui lui ressemblent le plus (ses "plus proches voisins") et on prend leur moyenne. Un passager dont on ignore l'âge mais qui voyage en 1re classe, seul, avec un billet cher, se verra attribuer l'âge moyen de passagers similaires, plutôt que la médiane globale.

```
from sklearn.impute import KNNImputer

# On donne au KNN des colonnes numériques corrélées à l'âge
sous_ensemble = df[["age", "pclass", "fare", "sibsp", "parch"]]

imputer_knn = KNNImputer(n_neighbors=5)
rempli = imputer_knn.fit_transform(sous_ensemble)
print("Âge reconstruit pour la ligne 5 :", round(rempli[5, 0], 1))
```

Devrait afficher quelque chose comme :

```
Âge reconstruit pour la ligne 5 : 30.4
```

C'est quoi le KNN, au fait ? "K-Nearest Neighbors", les k plus proches voisins. Le principe : "dis-moi qui te ressemble, j'en déduis ta valeur". On le reverra comme algo de classification en J3 ; ici il sert juste à boucher les trous intelligemment.

Étape 3 : mesurer l'impact. Ne jamais imputer en aveugle. Comparez la distribution avant/après : si imputer décale la moyenne ou écrase la variance, vous avez déformé la donnée.

```
age_brut = df["age"].dropna()
print(f"Avant : moyenne={age_brut.mean():.1f}, écart-type={age_brut.std():.1f}")
print(f"Médiane: moyenne={pd.Series(age_rempli.ravel()).mean():.1f}, "
      f"écart-type={pd.Series(age_rempli.ravel()).std():.1f}")
```

Vous verrez l'écart-type **baisser** après imputation par la médiane : normal, on a empilé 177 valeurs identiques au centre, donc on a artificiellement réduit la dispersion. C'est le prix de l'imputation simple, et c'est exactement pour ça que le KNN existe.

Manche d'observation : changez `strategy="median"` en `"mean"`, relancez la comparaison avant/après. La moyenne bouge-t-elle plus ou moins qu'avec la médiane ? Puis tentez `n_neighbors=1` puis `15` sur le KNN. Plus de voisins, est-ce plus stable ou plus lissé ?

On pousse plus loin

Un cadre qui fait la différence entre boucher des trous au pif et le faire en pro, la **typologie de Rubin** (du statisticien Donald Rubin). Pourquoi une donnée est-elle manquante ? Trois cas :

- **MCAR (Missing Completely At Random)** : le trou est purement aléatoire, sans lien avec quoi que ce soit. Un capteur qui bug au hasard. C'est le cas gentil, l'imputation simple suffit.
- **MAR (Missing At Random)** : le trou dépend d'une autre variable observée. Les hommes répondent moins souvent à la question "âge" que les femmes, par exemple. On peut le corriger en s'appuyant sur les autres colonnes (le KNN brille ici).
- **MNAR (Missing Not At Random)** : le trou dépend de la valeur manquante elle-même. Les très hauts revenus refusent de déclarer leur revenu. Le cas vicieux : le trou porte de l'information, et l'imputer cache un signal.

Le réflexe pro : avant d'imputer, demandez-vous **pourquoi** c'est manquant. Sur le Titanic, `deck` est massivement manquant pour les 3e classes : ce n'est pas du hasard (MNAR/MAR), l'absence de cabine attribuée est elle-même une info sociale. Boucher ça à la moyenne serait une faute.

Le piège éthique : supprimer les lignes incomplètes peut effacer toute une sous-population. Si les trous touchent surtout une catégorie de clients (les plus précaires remplissent moins de champs), votre `dropna()` les fait disparaître du modèle, qui devient aveugle à eux. On rebranche les lunettes "biais" de J1 : le nettoyage n'est jamais neutre.

Pour creuser : la doc [pandas "Working with missing data"](#) pour la manipulation concrète, et la page [scikit-learn sur l'imputation](#) pour aller vers le `IterativeImputer`, encore plus fin que le KNN.

Les trous sont bouchés. Mais une donnée présente peut être tout aussi traître qu'une donnée absente : la valeur aberrante, celle qui est là, bien visible, et complètement fausse (ou vraie mais monstrueuse). Au tour des outliers.

5 - Les valeurs aberrantes (outliers)

Un client qui a 207 ans. Un appartement à 0 euro. Une transaction de 4 millions sur un compte étudiant. Ces valeurs **extrêmes** s'appellent des outliers, et elles peuvent ruiner un modèle : une seule valeur monstrueuse tire la moyenne, fausse une régression, écrase l'échelle de toutes les autres. Mais attention, piège : un outlier n'est pas toujours une erreur. Parfois, c'est le cas le plus intéressant du dataset.

L'essentiel des outliers

Détecter visuellement : le boxplot. L'outil de base, inventé par le statisticien [John Tukey](#) dans les années 70. Il résume une distribution en cinq nombres et affiche les points suspects au-delà des "moustaches".

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("titanic")

sns.boxplot(x=df["fare"])
plt.title("Prix du billet : la boîte et ses outliers")
plt.show()
```

La boîte contient la moitié centrale des passagers ; les points isolés à droite sont les billets hors normes (des suites de luxe). Visuellement, en deux secondes, vous voyez qu'il y a un souci à droite.

Détecter par le calcul : la règle de l'IQR. Pour automatiser, on chiffre ce que le boxplot montre. L'**IQR (Interquartile Range, écart interquartile)** est la largeur de la boîte : la distance entre le 1er quartile (Q1, 25% des données en dessous) et le 3e quartile (Q3, 75% en dessous). Tout ce qui dépasse de 1,5 fois l'IQR au-delà des bords est un outlier.

```

Q1 = df["fare"].quantile(0.25)
Q3 = df["fare"].quantile(0.75)
IQR = Q3 - Q1

borne_basse = Q1 - 1.5 * IQR
borne_haute = Q3 + 1.5 * IQR

outliers = df[(df["fare"] < borne_basse) | (df["fare"] > borne_haute)]
print(f"Bornes 'normales' : [{borne_basse:.1f}, {borne_haute:.1f}]")
print(f"Nombre d'outliers détectés : {len(outliers)}")
print(f"Billet le plus cher : {df['fare'].max():.1f}")

```

Devrait afficher quelque chose comme :

```

Bornes 'normales' : [-26.7, 65.6]
Nombre d'outliers détectés : 116
Billet le plus cher : 512.3

```

Détecter par le z-score. Une autre approche : à combien d'écarts-types de la moyenne se trouve chaque point ? Au-delà de 3 écarts-types, on parle d'outlier. À utiliser surtout si la donnée est à peu près en cloche (loi normale).

```

from scipy import stats
import numpy as np

fare = df["fare"].dropna()
z = np.abs(stats.zscore(fare))
print(f"Points à plus de 3 écarts-types : {(z > 3).sum()}")

```

C'est quoi SciPy ? [SciPy](#) prolonge NumPy avec une grosse trousse scientifique (stats, optimisation, traitement du signal). On lui emprunte juste son calcul tout fait du z-score.

IQR ou z-score ? L'IQR est robuste : il se fiche de la forme de la distribution et n'est pas lui-même tiré par les extrêmes. Le z-score suppose une distribution en cloche et, ironie, la moyenne et l'écart-type qu'il utilise sont eux-mêmes faussés par les outliers. En cas de doute, IQR.

Décider quoi en faire. C'est ici que se joue le métier, et il n'y a pas de réponse automatique :

- **Supprimer** si c'est clairement une erreur de saisie (un âge de 207 ans).
- **Plafonner (capping/winsorisation)** : on ramène les valeurs extrêmes à la borne haute, pour garder la ligne sans la laisser tout dominer.
- **Garder et signaler** si l'outlier est réel et important. En détection de fraude, l'outlier EST la cible ! Le supprimer reviendrait à jeter exactement ce qu'on cherche.

Sur le Titanic, les billets à 512 livres sont de vraies suites de luxe, pas des erreurs. Les supprimer effacerait les passagers les plus riches, qui avaient justement le plus de chances de survivre. Un outlier réel raconte quelque chose.

Bougez les bornes : changez le `1.5` de la règle IQR en `3.0` (plus tolérant) puis `1.0` (plus sévère). Combien d'outliers à chaque fois ? Vous sentez que "outlier" n'est pas une vérité absolue, c'est un seuil que VOUS choisissez.

Creusons

Le piège classique, et il est grave : **supprimer les outliers améliore les métriques en apparence et casse le modèle en réalité**. En fraude, en panne industrielle, en maladie rare, les cas extrêmes sont rares ET précieux. Un modèle entraîné après avoir "nettoyé" tous les outliers n'aura jamais vu une fraude et sera incapable d'en détecter une. C'est le revers exact de la médaille : parfois la valeur extrême est du bruit à virer, parfois c'est le signal à protéger. Seule la compréhension métier (la phase 1 de CRISP-DM, encore elle) tranche.

Bon à savoir : il existe des algos dédiés à la détection d'anomalies, comme l'[Isolation Forest](#) de scikit-learn, qui repère les outliers en multidimensionnel (un point peut être normal sur chaque colonne prise seule, mais aberrant par sa combinaison). Hors programme pour nous, mais c'est tout un pan du ML non-supervisé.

💡 **Une graine pour plus tard :** la détection d'anomalies est un excellent sujet de projet et un marché entier (cybersécurité, maintenance, finance). Si le combo "non-supervisé + cas réels rares" vous parle, c'est une spécialité qui recrute.

On a nettoyé colonne par colonne. Mais une donnée, c'est aussi des relations ENTRE les colonnes. Et certaines de ces relations sont des pièges. Pour les voir, il nous faut plus de colonnes que le Titanic n'en offre, alors changeons de terrain.

6 - Corrélations et multicolinéarité

Le Titanic était parfait pour apprendre à nettoyer, mais avec ses 7 ou 8 colonnes, il est trop maigre pour la suite. Pour parler de corrélations et de réduction de dimensions, il faut de la largeur. On reprend donc un vieux camarade d'hier : le dataset du **cancer du sein**, qu'on avait à peine regardé en J1. Aujourd'hui, on l'autopsie. 30 colonnes de mesures, de quoi voir des vraies relations.

Le coeur du sujet

La corrélation, on l'a croisée hier : un nombre entre -1 et 1 qui dit si deux variables bougent ensemble. Proche de 1, elles montent ensemble ; proche de -1, l'une monte quand l'autre descend ; proche de 0, aucun lien linéaire. La façon de tout voir d'un coup, c'est la **heatmap** (carte de chaleur) des corrélations.

```
from sklearn.datasets import load_breast_cancer
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = load_breast_cancer(as_frame=True)
df = data.frame

# On regarde les 10 premières mesures pour que la carte reste lisible
cols = data.feature_names[:10]
matrice = df[cols].corr()
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(matrice, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Corrélations entre mesures de tumeurs")
plt.tight_layout()
plt.show()
```

Vous allez voir des carrés rouge foncé hors de la diagonale : `mean radius`, `mean perimeter` et `mean area` sont corrélées à plus de 0,99. Logique ! Le périmètre et l'aire d'une tumeur sont des fonctions quasi directes de son rayon. Ce sont des variables **redondantes** : elles disent trois fois la même chose.

Le rappel qui sauve : corrélation n'est pas causalité. Deux variables corrélées peuvent n'avoir aucun lien de cause à effet, juste une cause commune ou un pur hasard. Le site [Spurious Correlations](#) collectionne des corrélations absurdes (la consommation de fromage et les décès par drap emmêlé). À ressortir en réunion quand quelqu'un confond les deux.

La multicolinéarité, le vrai problème. Quand deux features ou plus sont fortement corrélées entre elles, on parle de multicolinéarité, et ça **déstabilise les modèles linéaires** (régression).

Features - le mot anglais pour nos variables d'entrée, les colonnes qui servent à prédire

=> On dit indifféremment feature, variable ou colonne

Par rapport à cette déstabilisation, voilà l'intuition importante à sentir :

Imaginez prédire un poids à partir de la taille en centimètres ET de la taille en pouces. Ça c'est la même info deux fois. Le modèle peut écrire "+10 pour les cm, 0 pour les pouces", ou "0 pour les cm, +25 pour les pouces", ou mille autres combinaisons qui donnent la même prédiction.

Du coup, les coefficients deviennent **instables** : si on ajoute une seule ligne de données ils valsent. Leur signe peut même s'inverser. La **prédiction reste correcte**, mais l'**interprétation s'effondre**. Or hier on a vu qu'en banque ou en assurance, on a besoin d'expliquer les coefficients (le droit à l'explication). La multicolinéarité tue cette explicabilité.

Mesurer la multicolinéarité : le VIF. Le **VIF (Variance Inflation Factor, facteur d'inflation de la variance)** chiffre le problème, feature par feature. Pour une variable, on essaie de la prédire à partir de toutes les autres : si on y arrive très bien, c'est qu'elle est redondante.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

X = df[["mean radius", "mean perimeter", "mean area", "mean smoothness"]]

vif = pd.DataFrame()
vif["variable"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)
```

Devrait afficher des valeurs énormes pour les trois variables redondantes :

	variable	VIF
0	mean radius	3806.1
1	mean perimeter	3786.4
2	mean area	420.5
3	mean smoothness	1.1

La règle de lecture : Si **VIF > 5 (parfois 10)**, alors = **multicolinéarité problématique**.

Ici 3806, c'est gigantesque : `mean radius` est presque parfaitement prédite par les autres. La parade est simple : on garde une seule des variables redondantes et on jette les autres. On ne perd quasiment aucune information (elles disaient la même chose) et on rend le modèle stable.

C'est quoi [statsmodels](#) ? Une bibliothèque Python orientée statistiques classiques (tests, régressions détaillées, diagnostics), complémentaire de scikit-learn qui, lui, est orienté prédiction. Le VIF est un diagnostic stat, d'où statsmodels.

Repérez les jumelles : affichez la heatmap sur les 10 features, repérez à l'oeil deux variables corrélées à plus de 0,9. Calculez leur VIF. Puis supprimez-en une et recalculez : le VIF de celle qui reste s'effondre-t-il ? C'est le geste exact que vous referez cet après-midi.

On va plus loin

Trois solutions à la multicolinéarité, par ordre de finesse :

- **Supprimer** une des variables redondantes. Simple, efficace, lisible. Le réflexe par défaut.
- **Régulariser** avec une régression Ridge, qui rétrécit les coefficients corrélés au lieu de les laisser exploser. (On croise Ridge/LASSO en J3.)
- **Transformer** avec la PCA : elle fabrique de nouvelles variables décorréliées par construction. C'est justement notre dernière section, et ça ferme la boucle : la PCA est, entre autres, une réponse à la multicolinéarité.

Pour creuser : [StatQuest, "Principal Component Analysis \(PCA\), Step-by-Step"](#) (~21 min) prépare exactement la prochaine section, et la chaîne [StatQuest](#) en général est la meilleure pédagogie ML en vidéo. À mater plus tard.

On sait voir les variables redondantes. Question miroir, et c'est l'autre face du tri : parmi toutes nos colonnes, lesquelles servent vraiment à prédire la cible ? Toutes ne se valent pas.

7 - Les variables les plus discriminantes

On a 30 colonnes. Est-ce qu'on en a besoin de 30 pour prédire si une tumeur est maligne ? Sûrement pas. Certaines portent presque tout le pouvoir prédictif, d'autres sont du bruit. Savoir les trier, c'est la **sélection de features**, et c'est un gain double : un modèle plus simple, plus rapide, plus explicable, et souvent plus performant (moins de bruit, moins de surapprentissage).

Ce qu'il faut retenir

Attention à ne pas confondre avec la section précédente : là, on cherchait les variables redondantes ENTRE elles (corrélées les unes aux autres). Ici, on cherche les variables liées à la **cible** (corrélées au label à prédire). Deux questions différentes.

Approche 1 : la corrélation à la cible. La plus directe pour du quantitatif. On regarde quelles features sont les plus corrélées au label.

```
from sklearn.datasets import load_breast_cancer
import pandas as pd

data = load_breast_cancer(as_frame=True)
df = data.frame # inclut la colonne 'target' (0 = malin, 1 = bénin)

# Corrélation de chaque feature avec la cible, triée par force
corr_cible = df.corr()["target"].drop("target").abs().sort_values(ascending=False)
print("Top 5 des features les plus liées à la cible :")
print(corr_cible.head())
```

Devrait afficher quelque chose comme :

```
worst concave points    0.79
worst perimeter         0.78
worst radius            0.78
mean concave points     0.77
mean concavity          0.70
```

Ces variables-là portent le signal. Une feature corrélée à 0,05 avec la cible, elle, ne sert quasiment à rien.

Approche 2 : l'information mutuelle. La corrélation ne capte que les liens **linéaires**. Une relation en U (forte aux extrêmes, faible au milieu) aura une corrélation proche de 0 alors qu'elle est très informative.

L'[information mutuelle](#) capte aussi les liens non linéaires.

```
from sklearn.feature_selection import mutual_info_classif

X = df.drop(columns="target")
y = df["target"]

mi = mutual_info_classif(X, y, random_state=42)
mi_serie = pd.Series(mi, index=X.columns).sort_values(ascending=False)
print(mi_serie.head())
```

Approche 3 : l'importance selon un modèle. Un Random Forest (croisé hier, vu en détail en J3) calcule gratuitement à quel point chaque feature a aidé à décider. C'est souvent la mesure la plus fiable en pratique.

```
from sklearn.ensemble import RandomForestClassifier

modele = RandomForestClassifier(n_estimators=100, random_state=42)
modele.fit(X, y)

importances = pd.Series(modele.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False).head())
```

Filtre, wrapper, ou embedded ? Trois grandes familles de sélection : les **filtres** jugent chaque feature isolément, vite (corrélation, information mutuelle) ; les **wrappers** testent des sous-ensembles en réentraînant un modèle, lent mais précis ; les **embedded** font la sélection pendant l'entraînement (l'importance du Random Forest, ou le LASSO qui annule les coefficients inutiles). Bon à savoir pour situer une méthode.

Comparez les classements : affichez le top 5 selon les trois approches. Tombent-elles d'accord ? Souvent oui sur les grands gagnants, mais l'ordre diffère. Si une feature est top en information mutuelle mais nulle en corrélation, qu'est-ce que ça vous dit sur la forme de sa relation à la cible ? (Indice : non linéaire.)

Pour aller plus loin

Le piège vicieux à connaître : la sélection de features peut **fuir**. Si vous choisissez vos features en regardant TOUT le dataset (train + test mélangés), vous laissez le test influencer un choix de modélisation : c'est une fuite de données, cousine de celle vue hier avec le scaler. La règle est la même et elle est absolue : **toute décision de preprocessing se prend sur le train seul**. On y revient en force cet après-midi, c'est le fil conducteur du TD.

Autre nuance pro : sélectionner les features une par une rate les **interactions**. Deux variables peuvent être nulles séparément mais très prédictives ensemble (la combinaison "âge jeune + montant élevé" en fraude). Les méthodes par modèle (Random Forest) attrapent ça ; les filtres simples, non. D'où l'intérêt de croiser plusieurs approches.

On sait jeter les colonnes inutiles une par une. Mais il existe une approche radicalement différente : au lieu de choisir parmi les colonnes existantes, on en fabrique de nouvelles, moins nombreuses, qui résument tout. C'est la PCA, et c'est le sommet de la matinée.

8 - La réduction de dimensions : la PCA

Imaginez 100 colonnes. Impossible à visualiser, lourd à entraîner, plein de redondances. Et si on pouvait compresser ces 100 colonnes en 2 ou 3, en gardant l'essentiel de l'information ? C'est exactement ce que fait l'**Analyse en Composantes Principales (ACP en français, PCA en anglais)**. C'est la star de la réduction de dimensions, et c'est du non-supervisé : elle ne regarde jamais la cible.

L'essentiel de la PCA

L'intuition d'abord, le code ensuite. Pas de formules lourdes, je veux que vous SENTIEZ ce qui se passe.

Imaginez un nuage de points en 3D, mais étiré comme une frite : très allongé dans une direction, un peu moins dans une autre, presque plat dans la troisième. La PCA cherche les **axes de plus grande dispersion** de ce nuage :

- La **1re composante principale** : la direction où les points sont le plus étalés (la longueur de la frite). C'est l'axe qui capture le plus d'information.
- La **2e composante** : la direction de plus grande dispersion restante, perpendiculaire à la première (la largeur).
- Et ainsi de suite.

Ensuite, on **projette** les points sur les premiers axes seulement, et on jette les derniers (l'épaisseur quasi nulle de la frite, qui ne portait presque rien). On passe de 30 dimensions à 2, en perdant le minimum d'information.

Et les "vecteurs propres" dans tout ça ? Si vous lisez la théorie, vous tomberez sur "vecteurs propres" et "valeurs propres". L'idée sans les maths : les composantes principales SONT les vecteurs propres de la matrice de covariance, et chaque valeur propre dit combien de variance son axe capture. Retenez l'intuition (axes de plus grande dispersion) ; la dérivation complète est hors de notre cours. Si l'algèbre linéaire vous tente, la vidéo [Eigenvectors and eigenvalues de 3Blue1Brown](#) (~17 min) la rend visuelle et limpide, c'est la meilleure intro qui existe.

Le point crucial : la PCA exige des données mises à l'échelle. Comme elle raisonne en dispersion, une variable en euros (qui varie de 0 à 100000) écraserait une variable en années (0 à 100). On scale TOUJOURS avant une PCA.

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target # 30 colonnes

# Étape obligatoire : mettre à l'échelle (moyenne 0, écart-type 1)
X_scaled = StandardScaler().fit_transform(X)

# On demande 2 composantes pour visualiser en 2D
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)

print("Forme avant PCA :", X_scaled.shape)
print("Forme après PCA :", X_2d.shape)
```

Devrait afficher :

```
Forme avant PCA : (569, 30)
Forme après PCA : (569, 2)
```

30 colonnes devenues 2. Et le miracle, c'est qu'on peut maintenant tout voir sur un plan :

À manipuler à la souris : [Principal Component Analysis explained visually](#) (setosa.io) laisse bouger le nuage de points et voir les axes principaux pivoter en direct. Cinq minutes là-dessus et vous SENTEZ la PCA mieux qu'avec n'importe quelle formule.

```
plt.figure(figsize=(8, 6))
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y, cmap="coolwarm", alpha=0.6)
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.title("569 tumeurs projetées de 30D vers 2D")
plt.colorbar(label="0 = malin, 1 = bénin")
plt.show()
```

Vous verrez les tumeurs malignes et bénignes former deux paquets assez nets, alors que la PCA n'a JAMAIS vu les étiquettes. La structure était bien dans la donnée.

La variance expliquée : combien a-t-on gardé ? La question qui tue : en compressant, combien d'information a-t-on perdu ? La PCA le dit exactement.

```
import numpy as np

# On refait une PCA sans limiter le nombre de composantes
pca_full = PCA().fit(X_scaled)
variance_cumulee = np.cumsum(pca_full.explained_variance_ratio_)

print(f"Variance gardée avec 2 composantes : {variance_cumulee[1]:.1%}")
print(f"Variance gardée avec 5 composantes : {variance_cumulee[4]:.1%}")
print(f"Composantes pour atteindre 95% : {np.argmax(variance_cumulee >= 0.95) + 1}")
```

Devrait afficher quelque chose comme :

```
Variance gardée avec 2 composantes : 63.2%
Variance gardée avec 5 composantes : 84.7%
Composantes pour atteindre 95% : 10
```

Traduction : avec seulement 2 composantes (sur 30), on garde déjà 63% de l'information. Avec 10, on atteint 95%. On peut donc remplacer 30 colonnes par 10 en ne perdant que 5%. C'est ça, la puissance de la PCA.

Le "scree plot" ? Le graphe de la variance expliquée par composante (ou cumulée). On cherche le "coude", l'endroit où ajouter une composante n'apporte presque plus rien. C'est là qu'on coupe.

Faites varier la coupe : changez le seuil de 0.95 à 0.80 puis 0.99. Combien de composantes faut-il à chaque fois ? Vous verrez le compromis en direct : plus on veut d'information, plus on garde de dimensions, moins on compresse.

Approfondissons

Ce qu'on PERD avec la PCA, et c'est majeur : l'interprétabilité. Les composantes principales sont des mélanges de toutes les features d'origine. La "composante 1" n'est pas "le rayon de la tumeur", c'est "0,22 x rayon + 0,21 x périmètre + 0,2 x aire + ...". Impossible de dire à un médecin "votre risque vient de la composante 1". On rebranche le dilemme de J1 : performance contre explicabilité. En banque, en santé, en justice, la PCA peut être disqualifiée pour cette seule raison, même si elle améliore le score. Le bon outil dépend du besoin, jamais de la mode.

Un peu de culture qui pose le contexte : la PCA a été inventée par **Karl Pearson en 1901**, puis redéveloppée par Harold Hotelling dans les années 30. Plus d'un siècle, et toujours partout. C'est aussi une arme contre la **"malédiction de la dimensionnalité"** (terme du mathématicien Richard Bellman) : en très haute dimension, tous les points finissent par sembler équidistants et les algos se noient. Réduire les dimensions, c'est leur redonner de l'air.

Pour les curieux : la PCA a une application visuelle bluffante, les **eigenfaces** (visages propres) : on peut reconstruire approximativement n'importe quel visage humain comme une combinaison de quelques dizaines de "visages de base". C'est de la PCA appliquée à des photos, et c'est une des premières techniques de reconnaissance faciale. Tapez "eigenfaces" pour voir les images, c'est saisissant.

Pour creuser : au-delà de la PCA, il existe des techniques de visualisation non linéaires comme [t-SNE](#) et [UMAP](#), redoutables pour faire apparaître des clusters cachés en 2D. Hors programme, mais ce sont les outils que vous verrez dans tous les notebooks d'exploration sérieux.

Attention au piège : l'article interactif [How to Use t-SNE Effectively](#) (Distill) montre à quel point ces cartes peuvent mentir si on lit mal leurs paramètres. À l'inverse de la PCA, la distance entre deux paquets sur un t-SNE ne veut souvent rien dire.

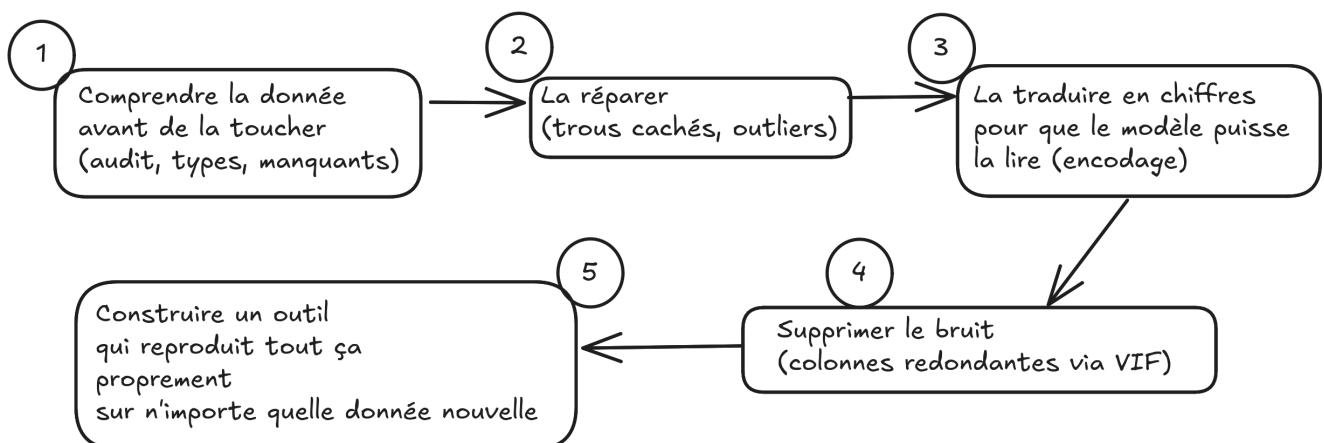
Voilà l'arsenal complet du matin : on sait inspecter, typer, encoder, boucher, nettoyer, trier et compresser une donnée. Cet après-midi, on arrête de regarder et on met les mains dedans, sur un vrai jeu crasseux.

À votre tour

Ce matin, on a manipulé des bouts de donnée chacun dans son coin. Cet après-midi, vous prenez UN dataset sale, réaliste, et vous le menez de l'état "brut et inexploitable" à l'état "propre, prêt à modéliser", de bout en bout. C'est exactement le boulot qu'on vous paiera à faire, et celui qui occupe 80% du temps d'un data scientist.

Ce que vous allez faire (pour rappel) :

Le Vrai Job d'un Data Scientist



Le fil rouge ici : chaque erreur qu'on peut faire (inventer un ordre, laisser fuiter le test, recréer des colonnes différentes selon la donnée) a une conséquence concrète sur le modèle. Le cours vous les fait rencontrer une par une, dans l'ordre où elles se présentent dans la vraie vie.

Le projet

Vous travaillez sur le **Telco Customer Churn**, un dataset télécom classique : environ 7000 clients d'un opérateur, une vingtaine de colonnes (ancienneté, type de contrat, montant facturé, services souscrits...), et une cible : le client a-t-il résilié, oui ou non ? C'est de la donnée d'entreprise typique, avec tous les défauts du monde réel : des types incohérents, des trous cachés, des colonnes redondantes, du texte à encoder partout.

Le coeur du livrable, celui que tout le monde ramène : un notebook propre qui prend le CSV brut du Telco et produit un jeu de données nettoyé, encodé, sans fuite, prêt à passer dans l'Arène de demain (phases 0 à 7), avec un README qui justifie vos choix de nettoyage. Ça, c'est la cible que chacun atteint.

À partir de là, le travail se prolonge naturellement, et c'est là que la donnée devient vraiment vôtre :

- vous transformez votre notebook en **boîte à outils** réutilisable, un module de nettoyage que vous rejouerez tel quel demain dans l'Arène et vendredi sur le projet de groupe (phase 8) ;
- vous **crash-testez** cette boîte à outils sur un deuxième dataset que VOUS choisissiez, perso ou fun (phase 9), avant le grand bilan (phase 10).

Le tout poussé sur GitHub, au fil des phases. Personne ne plafonne : selon votre rythme, vous irez plus ou moins loin dans le prolongement, et c'est exactement le but.

Vous avez les signatures, les sorties attendues et les phases. À vous d'écrire le code. Bloquer fait partie du jeu : c'est là qu'on apprend pour de vrai.

Phase 0 : Récupérer la donnée et l'ouvrir

L'intendance d'abord. Le dataset Telco Churn est public sur [Kaggle](#). Téléchargez le CSV (`WA_Fn-UseC_-Telco-Customer-Churn.csv`), déposez-le dans votre environnement (un upload dans [Google Colab](#), le notebook gratuit de Google qui tourne dans le navigateur sans rien installer, ou à côté de votre notebook en local), et chargez-le.

```
import pandas as pd

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print("Forme :", df.shape)
print(df.dtypes)
df.head()
```

Avant de coder : votre repo GitHub est votre copie d'examen continue. On note les commits, pas seulement le rendu final. Committez après chaque phase, avec des messages clairs et atomiques (un commit = une étape). Pas de `git add .` à l'aveugle : ajoutez vos fichiers un par un. Premier commit ici : le notebook qui charge la donnée.

Phase 1 : L'audit qualité

Avant de toucher quoi que ce soit, on diagnostique. Écrivez une fonction qui produit un rapport de santé du dataset : dimensions, types, taux de manquants par colonne, et surtout l'équilibre de la cible (`Churn`).

```
def audit_qualite(df):
    """Affiche un rapport de santé du dataset.
    Doit montrer : forme, types, % de manquants par colonne (triés),
    et la répartition de la cible Churn (en valeur ET en pourcentage).
    """
    # TODO : afficher la forme (lignes, colonnes)
    # TODO : pour chaque colonne, le pourcentage de valeurs manquantes, trié décroissant
    # TODO : la répartition de la cible Churn en nombre et en %
    pass
```

Devrait afficher quelque chose comme :

```
Forme : (7043, 21)
Manquants détectés : 0 colonne (méfiance : des trous sont peut-être cachés, voir Phase 2)
Churn Non : 5174 (73.5%)
Churn Oui : 1869 (26.5%)
```

Checkpoint qualité. Testez votre audit sur 3 situations :

- cas normal : le dataset complet.
- cas limite : passez un dataset filtré à une seule classe de churn. Le rapport reste-t-il honnête (pas de division par zéro, pas de plantage) ?
- cas adversarial : la cible est-elle déséquilibrée (73/27) ? Votre audit le rend-il visible en un coup d'oeil ? Demain, ce déséquilibre piégera l'accuracy, alors il doit sauter aux yeux dès maintenant.

Phase 2 : La colonne piégée (types incohérents et trous cachés)

Le Telco a un piège célèbre : la colonne `TotalCharges` est stockée en texte (`object`) alors que ce sont des montants, parce que quelques lignes contiennent un espace " " au lieu d'un nombre. Des trous déguisés. Repérez-les, convertissez la colonne en numérique, et traitez les manquants ainsi révélés.

```
def reparer_total_charges(df):
    """Convertit TotalCharges en numérique et traite les trous révélés.
    Doit renvoyer le df réparé et afficher combien de trous ont été démasqués.
    """
    # TODO : convertir TotalCharges en numérique (pd.to_numeric, errors="coerce"
    #         transforme les valeurs illisibles en NaN)
    # TODO : compter et afficher combien de NaN sont apparus
    # TODO : imputer ces trous (médiane) ou décider de supprimer ces lignes : justifiez
    pass
```

Rappel notation : un commit ici, message du type `fix: conversion TotalCharges + imputation des trous cachés`. Et notez dans le README pourquoi vous avez choisi d'imputer plutôt que de supprimer (ou l'inverse).

Ce qui doit casser, et que vous devez gérer :

- happy path : la conversion passe, le type devient `float64`.
- edge case : que se passe-t-il si une colonne est à 100% non numérique (du texte pur) ? Votre fonction doit le détecter et refuser de la convertir, pas la transformer en colonne 100% NaN en silence.

- adversarial : imaginez une colonne `MonthlyCharges` où un petit malin a écrit "29,90" avec une virgule au lieu d'un point. Votre `to_numeric` la transforme en NaN sans broncher. Comment repèreriez-vous ce genre de corruption AVANT qu'elle ne contamine tout ?

Phase 3 : Encoder les catégorielles

Le Telco regorge de colonnes texte : `gender`, `Contract`, `PaymentMethod`, des tas de `Yes/No` ... Un modèle ne les lit pas. Encodez-les proprement, en distinguant nominal et ordinal comme ce matin.

```
def encoder_features(df):
    """Encode toutes les colonnes catégorielles.
    Doit renvoyer un df 100% numérique, prêt pour un modèle.
    """
    # TODO : repérer les colonnes catégorielles (dtype object)
    # TODO : pour les binaires Yes/No, un encodage simple 0/1
    # TODO : pour les nominales (PaymentMethod, Contract...), du One-Hot
    # TODO : décider du sort de customerID (indice : un identifiant n'est PAS une feature)
    pass
```

Devrait afficher, par exemple, un passage de 21 colonnes à une quarantaine après One-Hot (le Telco a beaucoup de colonnes à plusieurs modalités), toutes numériques.

On verrouille : committez. Et un piège à débusquer absolument :

- happy path : toutes les colonnes texte sont devenues numériques.
- edge case : `Contract` a 3 modalités (Month-to-month, One year, Two year). Est-ce nominal ou ordinal ? Argumentez votre choix d'encodage dans le README.
- adversarial : que fait votre One-Hot si la colonne contient une catégorie ultra-rare présente sur 1 seul client ? Et le `customerID`, si vous l'encodiez par erreur en One-Hot, combien de colonnes ça créerait ? (Comptez. C'est la leçon de l'explosion de dimensions, en vrai.)

Phase 4 : Traiter les valeurs aberrantes

Les colonnes numériques (`tenure`, `MonthlyCharges`, `TotalCharges`) ont-elles des outliers ? Détectez-les avec la règle IQR vue ce matin, visualisez-les en boxplot, et décidez d'une stratégie par colonne (garder, plafonner, supprimer), en la justifiant.

```
def detecter_outliers_iqr(df, colonne):
    """Renvoie les bornes IQR et le nombre d'outliers d'une colonne numérique.
    Doit renvoyer (borne_basse, borne_haute, nombre_outliers).
    """
    # TODO : calculer Q1, Q3, IQR
    # TODO : déduire les bornes (Q1 - 1.5*IQR, Q3 + 1.5*IQR)
    # TODO : compter les points hors bornes
    pass
```

Avant de trancher : sur le churn, un client avec une facture mensuelle énorme n'est PAS forcément une erreur, c'est peut-être votre client le plus rentable (ou celui qui va partir le plus fort). Pour chaque outlier détecté, posez-vous : erreur de saisie, ou cas réel précieux ? Documentez la décision. Trois vérifications minimum : une colonne au comportement normal, une colonne sans aucun outlier (votre fonction renvoie

bien 0 sans planter ?), et le cas où vous supprimeriez les outliers, mesurez combien de clients "churn" vous perdriez au passage. Si supprimer les outliers efface 5% des résiliations, c'est un signal d'alarme.

Phase 5 : Corrélations et chasse à la multicolinéarité

Tracez la heatmap des corrélations sur les colonnes numériques. Repérez les variables redondantes (indice : `tenure`, `MonthlyCharges` et `TotalCharges` ne sont pas indépendantes, le total c'est grosso modo le mensuel multiplié par l'ancienneté). Calculez les VIF, et décidez quelles colonnes supprimer.

```
def rapport_multicolinearite(df, colonnes_num):  
    """Affiche la heatmap des corrélations et le VIF de chaque colonne numérique.  
    Doit aider à décider quelle(s) colonne(s) redondante(s) supprimer.  
    """  
    # TODO : heatmap seaborn des corrélations  
    # TODO : tableau des VIF (statsmodels variance_inflation_factor)  
    # TODO : signaler les colonnes au VIF > 5  
    pass
```

Petit point repo : committez votre analyse. Et vérifiez :

- happy path : la heatmap s'affiche, les VIF sont calculés.
- edge case : deux colonnes parfaitement identiques (dupliquez-en une volontairement). Que vaut le VIF ? (Il devrait exploser vers l'infini, ou planter : gérez-le.)
- adversarial : si vous supprimez `TotalCharges` pour cause de VIF élevé, vérifiez que vous ne supprimez pas par accident une info que les deux autres ne portent pas. Recalculez les VIF après suppression : le problème a-t-il disparu ?

Phase 6 : Les variables qui prédisent vraiment le churn

Maintenant, le tri par pouvoir prédictif. Quelles features sont les plus liées à la résiliation ? Croisez deux approches (corrélation à la cible et importance via un Random Forest) et produisez un classement.

```
def features_discriminantes(df, cible="Churn"):  
    """Classe les features par pouvoir prédictif sur la cible.  
    Doit afficher un top 10 selon au moins deux méthodes.  
    """  
    # TODO : corrélation (ou information mutuelle) de chaque feature à la cible  
    # TODO : importance via un RandomForestClassifier  
    # TODO : afficher les deux classements côte à côte  
    pass
```

Réflexe commit : sauvegardez. Les deux méthodes sont-elles d'accord sur le podium ? Si `Contract` ou `tenure` dominant, ça raconte une histoire métier (les clients en contrat court et récents partent le plus). Notez l'interprétation business dans le README : un bon data scientist relie toujours les chiffres au métier.

Trois vérifications avant de passer à la suite :

- happy path : une feature manifestement liée au churn (type `Contract`) ressort bien en tête des deux classements.

- edge case : une colonne quasi constante, ou `customerID` si vous l'aviez gardée par erreur, finit-elle tout en bas (pouvoir prédictif nul) ? Si un identifiant ressort dans le top, c'est un bug de fuite déguisée.
- adversarial : si une feature est forte en information mutuelle mais faible en corrélation linéaire, qu'est-ce que ça vous dit sur la forme de sa relation au churn ? (Indice : non linéaire, et c'est exactement ce que la corrélation rate.)

Phase 7 : Split, scaling, et le grand piège de la fuite

Le coeur du métier, et l'erreur qui coûte le plus cher en entreprise. Vous allez assembler tout le preprocessing dans le BON ordre, en respectant la règle d'or : **tout ce qui s'apprend sur la donnée (moyennes d'imputation, bornes de scaling) s'apprend sur le TRAIN seulement**, jamais sur le test.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def split_et_scale_proprement(X, y):
    """Sépare train/test PUIS ajuste le scaler sur le train seul.
    Doit renvoyer X_train_scaled, X_test_scaled, y_train, y_test.
    """
    # TODO : train_test_split (avec stratify=y pour garder l'équilibre du churn)
    # TODO : ajuster le StandardScaler sur X_train SEULEMENT (fit_transform)
    # TODO : appliquer ce même scaler à X_test (transform, PAS fit_transform)
    pass
```

Démontrez la fuite, chiffres en main. Faites-le dans les deux sens et comparez :

- version honnête : scaler ajusté sur le train seul.
- version qui triche : scaler ajusté sur tout X AVANT le split.

Entraînez un modèle rapide (une régression logistique) dans les deux cas et comparez l'accuracy. Surprise probable : sur ce dataset, avec un simple `StandardScaler`, l'écart sera minuscule, peut-être nul. Et c'est une leçon en soi, pas un ratage : avec 7000 lignes, le train et le test ont quasiment la même moyenne et le même écart-type, donc le scaler ne fuite presque rien. Le vrai poison, ce sont les fuites qu'on verra plus tard : une **imputation** ou une **sélection de features** ajustées sur tout le jeu (le modèle hérite alors de stats du test). Toutes les fuites ne se valent pas. Mais le réflexe reste absolu : on ajuste TOUJOURS sur le train seul, et l'objet `Pipeline` de scikit-learn (croisé hier : l'objet qui enchaîne les étapes de preprocessing et le modèle en un seul bloc) rend cette triche impossible par accident. *Pour la voir vraiment exploser, poussez plus loin : refaites la démo en ajustant l'imputation des manquants sur tout X, et regardez l'écart se creuser.*

Avant de lancer : committez la version propre ET la démo de fuite, message du type `feat: pipeline preprocessing anti-leakage + démo data leakage`.

Phase 8 : La boîte à outils réutilisable

Regardez ce que vous venez de faire : sept fonctions, écrites une par une, dans un notebook. Ça marche pour le Telco. Mais demain dans l'Arène, et vendredi sur le projet de groupe, vous allez recharger une donnée sale et... tout réécrire ? Non. Un data scientist ne nettoie pas à la main à chaque projet : il se fabrique un **outil** une bonne fois, et il le transporte. C'est ce qu'on construit là : votre premier module de nettoyage réutilisable. Vous le rejouerez tel quel toute la semaine.

L'idée centrale, et elle rejoint la règle d'or de la phase 7 : un bon outil de nettoyage **apprend sur le train** (les médianes d'imputation, les catégories vues, le scaler) et **rejoue ces décisions à l'identique** sur n'importe quelle donnée nouvelle, sans jamais réapprendre. C'est exactement la logique `fit / transform` de scikit-learn. On va se coder une petite classe maison, `DataCleaner` (une classe Python à nous, qui imite le contrat `fit / transform` des transformateurs scikit-learn), pour le sentir de l'intérieur.

```
class DataCleaner:
    """Apprend le nettoyage sur le train, le rejoue à l'identique sur toute donnée
    nouvelle.
    Même philosophie que scikit-learn : fit() APPREND, transform() APPLIQUE, jamais
    l'inverse.
    """

    def fit(self, df):
        # TODO : APPRENDRE et mémoriser tout ce qui se calcule sur la donnée :
        # - la médiane de chaque colonne numérique (pour l'imputation)
        # - les catégories vues pour chaque colonne (pour un One-Hot stable)
        # - les colonnes à jeter (VIF élevé, identifiants comme customerID)
        # - le StandardScaler ajusté sur le train
        # Ici on ne transforme RIEN. On range juste ce qu'on a appris dans self.
        # Retourne self.
        return self

    def transform(self, df):
        # TODO : rejouer EXACTEMENT les décisions apprises au fit, sans rien recalculer.
        # Une catégorie jamais vue au fit ? On l'ignore, on ne crée pas de colonne
        surprise.
        # Retourne un DataFrame 100% numérique, avec les MÊMES colonnes qu'au fit.
        pass

    def fit_transform(self, df):
        return self.fit(df).transform(df)
```

Le piège que cet outil doit tuer : `pd.get_dummies` (vu ce matin) crée des colonnes **différentes** selon les catégories présentes dans la donnée qu'on lui passe. Sur le train vous obtenez 40 colonnes, sur le test une catégorie manque et vous en obtenez 39 : le modèle de demain refuse de tourner, ou pire, mélange les colonnes en silence. C'est pour ça que la version pro fige les colonnes au `fit` avec `OneHotEncoder(handle_unknown="ignore")` plutôt que `get_dummies`. `get_dummies`, c'est parfait pour explorer (ce matin) ; pour un pipeline qui tourne en prod, c'est un piège.

Cassez votre propre outil : trois tests, à écrire en `assert` (un test qui passe ne dit rien à l'écran, un test qui casse lève une erreur, c'est le but) :

```
cleaner = DataCleaner()
X_train_clean = cleaner.fit_transform(X_train)
X_test_clean = cleaner.transform(X_test)    # transform SEUL, surtout pas de fit ici

assert X_train_clean.isna().sum().sum() == 0          # plus aucun trou
assert list(X_train_clean.columns) == list(X_test_clean.columns) # train et test alignés
```

- happy path : `fit_transform(X_train)` renvoie un DataFrame sans trou, 100% numérique.

- edge case : passez à `transform` une seule ligne (`X_test.head(1)`). Votre outil tient-il, ou suppose-t-il qu'il y a plusieurs lignes pour recalculer une stat (ce qui serait déjà une fuite) ?
- adversarial : fabriquez un test qui contient une catégorie **absente du train** (par exemple un `PaymentMethod` bidon présent uniquement dans le test). Est-ce que `transform` garde exactement les mêmes colonnes que le train, ou est-ce qu'il en ajoute/supprime une en douce et casse le modèle de demain ? C'est LE bug le plus courant en preprocessing d'équipe, et votre `assert` sur les colonnes doit l'attraper.

La version industrielle. Une fois que vous avez senti le `fit/transform` à la main, regardez comment l'industrie l'écrit : avec un `Pipeline` (croisé hier) qui enchaîne les étapes, et un `ColumnTransformer` qui applique un traitement différent aux colonnes numériques et catégorielles, le tout en un seul objet qui rend la fuite **impossible par construction**. Reconstruisez votre nettoyage dans ce format : c'est celui que vous utiliserez demain et vendredi, et celui que vous verrez dans tout code pro.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# TODO : un pipeline numérique (imputation médiane -> scaling)
# TODO : un pipeline catégoriel (imputation mode -> OneHotEncoder handle_unknown="ignore")
# TODO : assembler les deux avec ColumnTransformer, selon le type de colonne
# Le tout s'utilise ensuite comme un seul objet : preprocesseur.fit(X_train),
# .transform(X_test)
```

Vérifiez que cette version industrielle produit bien la même chose que votre `DataCleaner` maison : même nombre de colonnes en sortie, et toujours zéro fuite (le `.fit()` ne voit que le train).

```
# preprocesseur : le ColumnTransformer assemblé juste au-dessus
X_train_v2 = preprocesseur.fit_transform(X_train)
X_test_v2 = preprocesseur.transform(X_test)

# Doit afficher le MÊME nombre de colonnes que votre DataCleaner, et des sorties alignées.
print("Colonnes (version sklearn) :", X_train_v2.shape[1])
assert X_train_v2.shape[1] == X_test_v2.shape[1] # train/test alignés, par construction
```

Envie d'aller vers du vrai test logiciel ? Vos `assert` sont déjà des tests. Le standard Python pour les industrialiser, c'est `pytest` : vous rangez vos `assert` dans des fonctions `test_*` et un seul `pytest` les lance toutes. Pas obligatoire aujourd'hui, mais c'est le réflexe qui sépare un notebook jetable d'un module qu'on réutilise sans peur.

Point de sauvegarde : sortez votre boîte à outils du notebook, dans un vrai fichier

`preprocessing_toolkit.py`, et committez-le à part (feat: `DataCleaner` réutilisable `fit/transform` + `tests`). Un module versionné, c'est ce que vous importerez demain. Le notebook explore, le module dure.

Phase 9 : Le crash-test, sur une data que VOUS choisissiez

Votre boîte à outils nettoie le Telco. Mais un outil ne vaut que s'il survit à de la donnée qu'il n'a jamais vue. On va donc lui mettre une vraie claque : vous prenez un **autre** dataset, vous lui jetez votre `DataCleaner` dessus, et vous regardez ce qui casse. Parce que ça... *va* casser : chaque dataset est sale à sa façon. Et c'est exactement comme ça qu'une lib de nettoyage mûrit, en encaissant des données de plus en plus variées.

Choisir vite, et ne pas sécher. Voici un menu prêt à l'emploi. Chaque dataset cache un piège précis, vu ce matin, mais avec une saleté qui lui est propre. Pas besoin de chercher pendant dix minutes : prenez celui qui vous fait inspirer le plus et chargez-le. Évidemment si vous avez votre propre idée pour faire cet exercice, elle est la bienvenue.

Dataset	Lien	Cible à prédire	Le piège qui vous attend
Spaceship Titanic	Kaggle	Passager téléporté (oui/non)	Colonne <code>Cabin</code> qui colle trois infos ("deck/num/side") : à découper (tidy data), et des trous partout
FIFA 21 (messy)	Kaggle	Valeur ou note d'un joueur	Montants en texte ("€103.5M"), taille "5'7\"", poids "159lbs" : le piège <code>TotalCharges</code> puissance dix
Mushroom	Kaggle	Comestible ou vénéneux	100% catégoriel : stress-test du One-Hot, et un "?" planqué dans <code>stalk-root</code>
Ramen Ratings	Kaggle	Note du ramen	Des "Unrated" cachés dans la colonne <code>Stars</code> numérique : le jumeau exact du piège Telco
Pima Diabetes	Kaggle	Diabétique (oui/non)	Des zéros impossibles (glucose ou IMC = 0) : des manquants déguisés en vraies valeurs (MNAR)
Pokémon	Kaggle	Légendaire (oui/non)	<code>Type 2</code> à moitié vide, un nom à jeter, et une cible ultra-déséquilibrée

Si vous hésitez plus de dix secondes : prenez **Spaceship Titanic**, c'est le plus proche du Telco, vous démarrez tout de suite. Vous voulez de l'exotique ? FIFA messy est le plus crasseux et le plus fun à déboguer.

Et si vous tenez à VOTRE donnée (export Spotify, Strava, relevé bancaire en CSV), allez-y : c'est encore plus parlant. Mais ne passez pas vingt minutes à la chercher ou à la formater à la main : si ça coince, repliez-vous sur le menu et avancez.

L'exercice : chargez le dataset, lancez votre `DataCleaner.fit_transform` dessus, et notez tout ce qui plante ou produit n'importe quoi. Puis généralisez votre outil pour qu'il encaisse cette nouvelle donnée sans qu'on lui réécrive du sur-mesure.

Le scénario "qu'est-ce qui pète ?" Votre objectif minimal : que votre boîte à outils sorte, de bout en bout, un DataFrame numérique sans trou sur ce dataset inconnu. Mais en chemin, trois murs vont vous tomber dessus, et chacun est une leçon :

Premier mur, le type qu'on n'avait jamais vu. Ce dataset a sûrement une **colonne d'un genre que le Telco n'avait pas** : une date, un nom en texte libre, une colonne `Cabin` composite "deck/num/side". Votre outil fait quoi dessus ? Probablement n'importe quoi. À vous de trancher : jeter, parser, ou en faire une feature, et de le documenter.

Deuxième mur, l'explosion silencieuse. Repérez une colonne à **très haute cardinalité** (le `Name` d'un Pokémon, la `Brand` d'un ramen, le nom d'un joueur FIFA). Si votre One-Hot lui tombe dessus sans garde-fou, comptez les colonnes créées : des centaines, parfois des milliers. C'est l'explosion de dimensions de ce matin, en vrai. Votre outil doit la détecter et refuser de One-Hot une colonne qui dépasse, disons, 50 modalités.

Troisième mur, le faux propre. Certains datasets cachent leur saleté : un `0` qui veut dire "manquant" (Pima), un "Unrated" texte au milieu de notes (Ramen). Votre `DataCleaner` les avale sans broncher et vous pourrit la donnée en silence. Le test qui sauve : après nettoyage, ressortez un `describe()` et demandez-vous si les min/max ont un sens physique (un IMC de 0, ça n'existe pas).

Et une question ouverte, sans réponse unique : parmi votre dataset et ceux de vos voisins, lequel était le plus "propre" (le moins d'interventions pour le rendre exploitable) ? Lequel était un cauchemar ? Cette intuition, "à quel point une donnée est sale avant même de modéliser", c'est un réflexe de pro que vous êtes en train de muscler.

On archive : committez votre crash-test (`feat: crash-test DataCleaner sur <dataset>`) et notez dans le README les ajustements que ce nouveau dataset a forcés. Et si ce dataset vous a plu, gardez-le sous le coude : vendredi, chaque équipe choisit son terrain de jeu pour le projet, et vous venez peut-être de trouver le vôtre, déjà à moitié nettoyé.

Phase 10 : Le bilan, et jusqu'où on peut pousser

Une phase sans ligne d'arrivée : il y a toujours mieux à faire. Trois chantiers, à mener autant que le temps le permet.

Le tableau de bord du nettoyage. Agrégez les métriques de toutes les phases dans un seul récapitulatif : nombre de colonnes avant/après, lignes supprimées, trous bouchés, colonnes redondantes virées, dimensions finales. Une vue d'ensemble de ce que votre pipeline a transformé. Quelque chose comme :

Étape	Avant	Après	
Colonnes	21	40	(après One-Hot)
Lignes	7043	7043	(imputation médiane, aucune perdue)
Trous cachés	11	0	
Colonnes au VIF > 5	2	0	

(Vos chiffres dépendront de vos choix : si vous aviez supprimé les 11 lignes au lieu d'imputer, ce serait 7032 lignes. C'est tout l'intérêt du tableau : il rend vos décisions visibles.)

Le comparatif des stratégies. Vous avez fait des choix (imputer par médiane, supprimer telle colonne, plafonner tel outlier). Mesurez-les : entraînez une régression logistique baseline sur deux versions de votre dataset (par exemple imputation médiane contre imputation KNN, ou avec contre sans suppression des colonnes redondantes) et comparez les résultats. Attention au piège tendu dès la Phase 1 : sur du 73/27, l'accuracy flatte (un modèle qui répond toujours "Non" fait déjà 73,5% sans rien comprendre). Regardez donc aussi combien de **vrais churners** chaque version récupère, pas seulement l'accuracy. Quelle stratégie de nettoyage donne le meilleur résultat ? C'est ça, prouver qu'un choix de preprocessing était bon. (Et c'est un avant-goût de J4 : la bonne métrique n'est presque jamais l'accuracy seule.)

La PCA en exploration. Appliquez une PCA sur vos features numériques scalées, projetez en 2D, colorez par churn. Voit-on les résiliers se séparer ? Combien de composantes pour garder 90% de la variance ? Ça vous donne une intuition de la redondance globale de votre donnée.

Le test ultime : est-ce que votre dataset final passerait dans le pipeline d'hier sans broncher ? Branchez-le sur un `.fit()` / `.predict()` rapide. S'il tourne, vous avez gagné : votre donnée est prête pour l'Arène de demain.

Peer review (relecture par deux)

Avant de figer votre version finale, je vous invite à échanger votre notebook avec un binôme. Vous vous relisez avec cette grille (ce n'est pas une note, c'est une discussion) :

- Est-ce que le preprocessing respecte l'ordre logique (audit, types, manquants, outliers, encodage, sélection, split, scaling) ?
- Est-ce qu'il y a une fuite de données cachée quelque part (un scaler ou un imputer ajusté avant le split) ?
- Les choix de nettoyage sont-ils justifiés dans le README, ou subis ?
- Sur les outliers et l'imputation, votre binôme a-t-il fait des choix **différents** des vôtres, tout aussi défendables ? Lesquels ?

Le but : voir qu'il n'y a pas une seule bonne réponse. Imputer par médiane ou par KNN, plafonner ou supprimer un outlier, garder ou jeter une colonne : tout se discute, et c'est la **justification** qui fait la qualité, pas le choix lui-même.

Avant de partir

Si votre notebook de nettoyage n'est pas sur GitHub, faites-le maintenant : c'est la trace de votre travail, et sans trace, pas d'évaluation. Votre dataset propre est le carburant de demain : on en aura besoin pour le grand Fight des algos.

Récapitulatif Jour 2

Ce qu'on a appris aujourd'hui

1. **Les familles de données** : structurées (notre terrain), semi-structurées (JSON), non structurées (texte, image), et le monde du flux (IoT, streaming).
2. **La nature statistique** : qualitatif (nominal, ordinal) contre quantitatif (discret, continu), et pourquoi ça dicte tout le traitement. Plus l'encodage : One-Hot pour le nominal, Ordinal pour l'ordinal.
3. **Les valeurs manquantes** : détecter (en %), corriger (suppression, imputation simple, KNN), et toujours mesurer l'impact. Se demander **pourquoi** c'est manquant.
4. **Les valeurs aberrantes** : boxplot, règle IQR, z-score. Et le discernement : un outlier est parfois une erreur, parfois le signal le plus précieux.
5. **Corrélations et multicolinéarité** : la heatmap pour voir, le VIF pour mesurer, et l'intuition des "variables jumelles" qui déstabilisent une régression.
6. **Les variables discriminantes** : trier les features par lien à la cible (corrélation, information mutuelle, importance d'un modèle).
7. **La PCA** : compresser 30 colonnes en 2 ou 3, lire la variance expliquée, et accepter le prix à payer, la perte d'interprétabilité.

Points d'attention pour demain

- Demain, on rentre enfin dans le vif : **les algorithmes**. Régression, arbres, forêts, boosting, SVM. Et surtout, on organise le **grand Fight de l'Arène** : tous ces algos qui s'affrontent sur un même problème.
- Et ils s'affronteront sur de la donnée PROPRE, celle que vous venez de préparer. Vous comprenez maintenant pourquoi ce jour était décisif : on ne nourrit pas un algo avec de la bouillie.
- Gardez en tête la règle d'or, elle reviendra toute la semaine : **toute décision de preprocessing s'apprend sur le train seul**. La fuite de données est l'erreur qui transforme un faux champion en désastre de production.